

컨테이너 실행 해보기

도커 엔진 시작 / 종료

컨테이너를 실행하지 않으면 컴퓨터의 리소스는 거의 차지하지 않는다.

도커엔진이 종료되면 컨테이너는 모두 정지된다.

도커엔진은 자동으로 구동되나 컨테이너는 자동으로 구동되지 않는다.

Docker Desktop을 이용해 Docker를 테스트할 것이므로, 실습을 시작하기 전에 Docker Desktop을 실행해 두어야 한다.

OS	도커엔진 시작	도커엔진 종료	자동실행설정
윈도우	시작 → Docker Desktop 선택	도커 데스크톱 창 닫기	기본으로 자동 구동
macOS	파인더 → '애플리케이션' 폴더 → Docker Desktop 선택	상태막대에서 도커 클릭 -> Quit Docker Desktop 선택	상태막대에서 도커 클릭 → Preference → Start Docker Desktop when you login 선택
리눅스	<code>sudo systemctl start docker</code>	<code>sudo systemctl stop docker</code>	<code>sudo systemctl enable docker</code>

도커 명령어 구조



image 제어 명령어

docker image 하위명령 옵션

주요 하위 커맨드

하위 커맨드	내용	생략 가능 여부	주요 옵션
pull	도커 허브 등의 리포지토리에서 이미지를 내려받음	O	거의 사용하지 않음
rm	도커 이미지를 삭제	*2	거의 사용하지 않음
ls	내려 받은 이미지의 목록을 출력	X	거의 사용하지 않음
build	도커 이미지를 생성	O	-t

*2: 생략형은 docker rmi

Container 제어 명령어

docker [container] 하위명령 옵션

주요 하위 커맨드

하위 커맨드	내용	생략 가능 여부	주요 옵션
start	컨테이너를 실행	O	-i
stop	컨테이너를 정지	O	거의 사용하지 않음
create	도커 이미지로부터 컨테이너를 생성	O	--name -e -p -v
run	도커 이미지를 내려받고 컨테이너를 생성해 실행함(다운 로드는 필요한 경우에만). docker image pull, docker container create, docker container start라는 세 개의 명령을 하나로 합친 것과 같다.	O	--name -e -p -v -d -i -t
rm	정지 상태의 컨테이너를 삭제	O	-f -v
exec	실행 중인 컨테이너 속에서 프로그램을 실행	O	-i -t
ls	컨테이너 목록을 출력	*1	-a
cp	도커 컨테이너와 도커 호스트 간에 파일을 복사	O	거의 사용하지 않음
commit	도커 컨테이너를 이미지로 변환	O	거의 사용하지 않음

*1: 생략형은 docker ps

volume 제어 명령어

docker volume 하위명령 옵션

주요 하위 커맨드

하위 커맨드	내용	생략 가능 여부	주요 옵션
create	볼륨을 생성	X	--name
inspect	볼륨의 상세 정보를 출력	X	거의 사용하지 않음
ls	볼륨의 목록을 출력	X	-a
prune	현재 마운트되지 않은 볼륨을 모두 삭제	X	거의 사용하지 않음
rm	지정한 볼륨을 삭제	X	거의 사용하지 않음

network 제어 명령어

docker network 하위명령어 옵션

주요 하위 커맨드

하위 커맨드	내용	생략 가능 여부	주요 옵션
connect	컨테이너를 도커 네트워크에 연결	X	거의 사용하지 않음
disconnect	컨테이너의 도커 네트워크 연결을 해제	X	거의 사용하지 않음
create	도커 네트워크를 생성	X	거의 사용하지 않음
inspect	도커 네트워크의 상세 정보를 출력	X	거의 사용하지 않음
ls	도커 네트워크의 목록을 출력	X	거의 사용하지 않음
prune	현재 컨테이너가 접속하지 않은 네트워크를 모두 삭제	X	거의 사용하지 않음
rm	지정한 네트워크를 삭제	X	거의 사용하지 않음

기타 명령어

그 밖의 상위 커맨드

상위 커맨드	내용
checkpoint	현재 상태를 일시적으로 저장한 후, 나중에 해당 시점의 상태로 되돌릴 수 있다. 현재 실험적 기능이다.
node	도커 스웜의 노드를 관리하는 기능
plugin	플러그인을 관리하는 기능
secret	도커 스웜의 비밀값 정보를 관리하는 기능
service	도커 스웜의 서비스를 관리하는 기능
stack	도커 스웜 또는 쿠버네티스에서 여러 개의 서비스를 합쳐 구성한 스택을 관리하는 기능
swarm	도커 스웜을 관리하는 기능
system	도커 엔진의 정보를 확인하는 기능

단독명령어

단독 커맨드	내용	주요 옵션
login	도커 레지스트리에 로그인	-u -p
logout	도커 레지스트리에 로그아웃	거의 사용하지 않음
search	도커 레지스트리를 검색	거의 사용하지 않음
version	도커 엔진 및 명령행 도구의 버전을 출력	거의 사용하지 않음

#도커 이미지 내려 받기

```
docker pull 이미지이름
```

#도커 이미지 생성하기

```
docker create 이미지이름
```

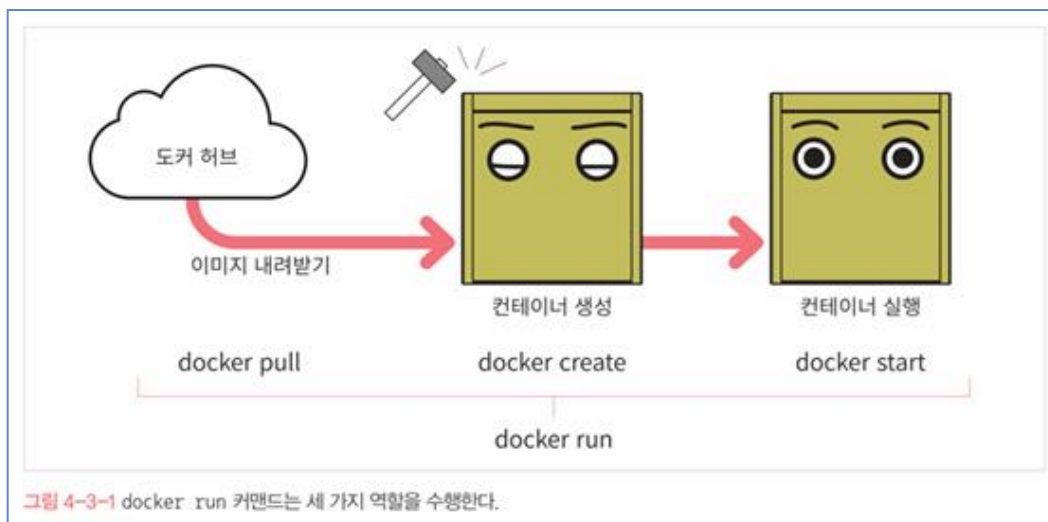
#도커 이미지 실행하기

```
docker start 이미지이름
```

도커 컨테이너 생성 및 실행 기능, 필요시 이미지 다운로드 기능도 수행

```
docker [container] run 옵션
```

- docker pull + docker create + docker start



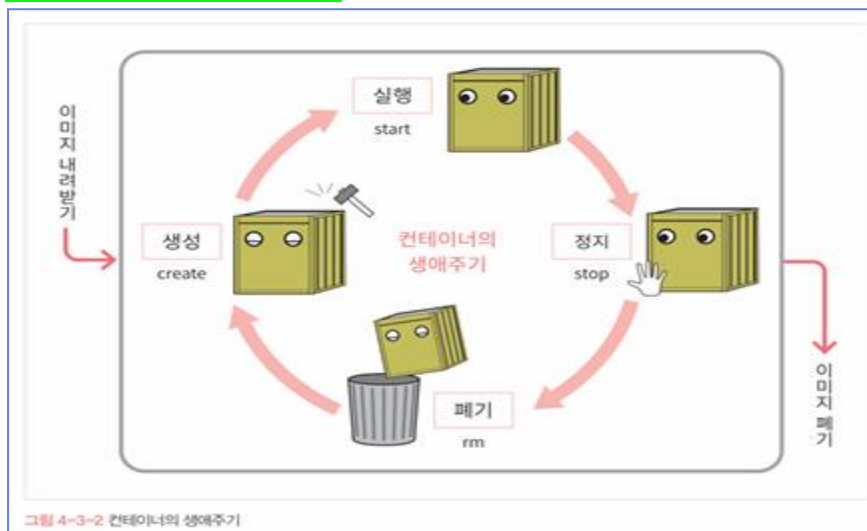
#컨테이너 정지 명령어

```
docker stop 컨테이너명
```

#컨테이너 삭제 명령어

```
docker rm 컨테이너명
```

도커컨테이너 생애주기



동작 중인 컨테이너는 삭제 불가능하기 때문에 컨테이너 정지 후 삭제 해야 한다.

실습하기 01

- docker Desktop을 실행한다.
- <https://hub.docker.com/> 에서 nginx를 검색한다.
- cmd창에서
 - docker images** - 현재 존재하는 images정보확인
 - docker container ps** - 현재 실행 중인 container정보 확인
 - docker run nginx** - nginx latest버전 pull + create + start

- 새로운cmd 창에서

docker images

docker container ps

- cmd창에서

docker stop 컨테이너이름

docker ps -a

docker rm 컨테이너이름

docker ps -a

- 실행중인 컨테이너를 중지 시킨다.

- 모든 컨테이너 정보 확인

- 컨테이너 삭제

- 모든 컨테이너 정보 확인

```

C:\Users\WKOSTA>docker run nginx
Unable to find image 'nginx:latest' locally
latest: Pulling from library/nginx
6e909acdb790: Download complete
5eaa34f5b9c2: Download complete
e7e0ca015e55: Download complete
417c4bccf534: Download complete
c22eb46e871a: Download complete
97f5c0f51d43: Download complete
373fe654e984: Download complete
Digest: sha256:124b44bfc9ccd1f3cedf4b592d4d1e8bddb78b51ec2ed5056c52d3692baeb
Status: Downloaded newer image for nginx:latest
/docker-entrypoint.sh: /docker-entrypoint.d/ is not empty, will attempt to p
/docker-entrypoint.sh: Looking for shell scripts in /docker-entrypoint.d/

```

```

Microsoft Windows [Version 10.0.19045.5608]
(c) Microsoft Corporation. All rights reserved.

C:\Users\WKOSTA>docker images
REPOSITORY TAG IMAGE ID CREATED SIZE
nginx latest 124b44bfc9cc 7 weeks ago 279MB

C:\Users\WKOSTA>docker ps
CONTAINER ID IMAGE COMMAND CREATED STATUS PORTS NAMES
7566a20eb8bd nginx "/docker-entrypoint..." About a minute ago Up About a minute 80/tcp silly_darwin

C:\Users\WKOSTA>docker container ps
CONTAINER ID IMAGE COMMAND CREATED STATUS PORTS NAMES
7566a20eb8bd nginx "/docker-entrypoint..." 2 minutes ago Up 2 minutes 80/tcp silly_darwin

C:\Users\WKOSTA>docker ps
CONTAINER ID IMAGE COMMAND CREATED STATUS PORTS NAMES
7566a20eb8bd nginx "/docker-entrypoint..." 9 minutes ago Up 9 minutes 80/tcp silly_darwin

C:\Users\WKOSTA>docker stop 7566a20eb8bd
7566a20eb8bd

C:\Users\WKOSTA>docker ps
CONTAINER ID IMAGE COMMAND CREATED STATUS PORTS NAMES

```

```

C:\Users\WKOSTA>docker ps -a
CONTAINER ID   IMAGE     COMMAND                  CREATED      STATUS      PORTS      NAMES
7566a20eb8bd   nginx    "/docker-entrypoint...."  17 minutes ago  Exited (0)  5 minutes ago
C:\Users\WKOSTA>docker rm 7566
7566
C:\Users\WKOSTA>docker ps -a
CONTAINER ID   IMAGE     COMMAND                  CREATED      STATUS      PORTS      NAMES
C:\Users\WKOSTA>

```

-a 옵션은 모든 container의 상태를 확인 할수 있다

nginx가 실행은 되었으나 호스트 pc에서 docker내부의 nginx를 실행 할 수 없다. 즉 **컨테이너는 기본적으로 외부에서 접근 불가능한(격리된) 상태로 실행된다.** 컨테이너와의 통신을 위해서는 **포트포워딩을 해야 한다.** **포트포워딩**이란 컨테이너 내부 네트워크의 포트와 호스트 포트(인터넷 포트)를 연결시켜 서로 통신할 수 있도록 열어주는 것을 말한다.

docker run 명령어시 옵션 설정해보자.

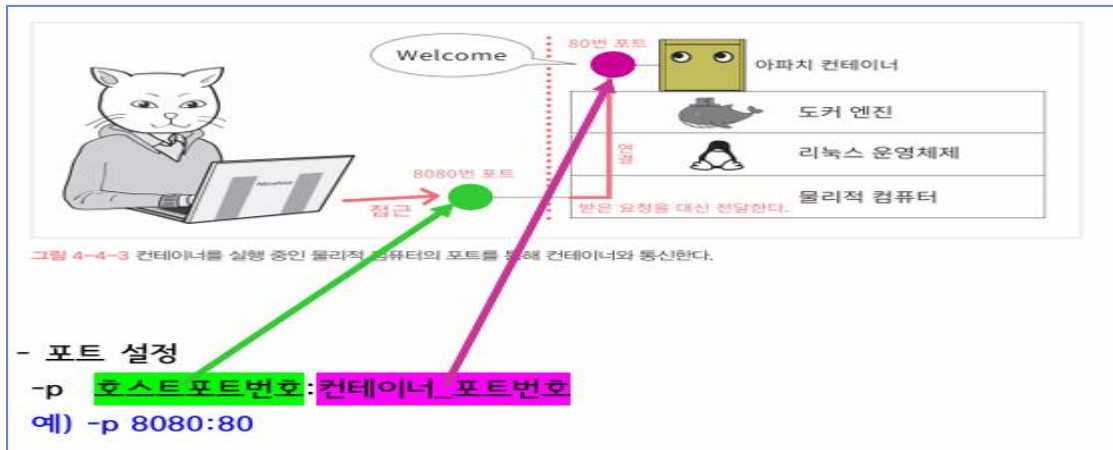
docker run 옵션 이미지 [인자]

주요 옵션

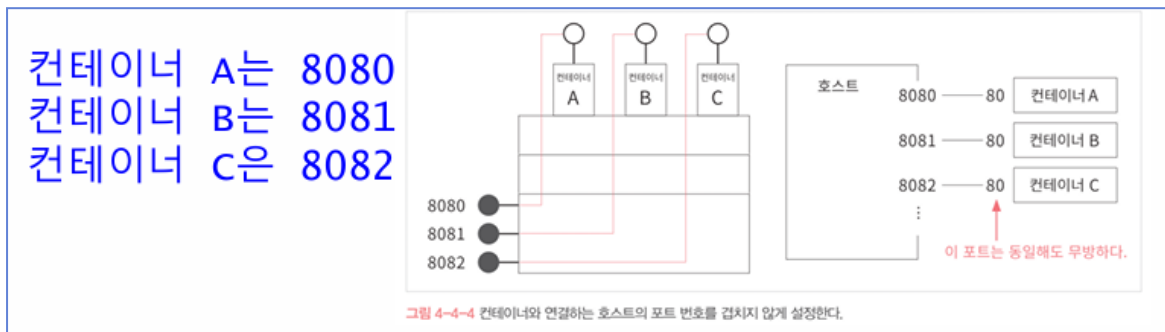
옵션 형식	내용
--name 컨테이너_이름	컨테이너 이름을 지정함
-p 호스트_포트번호:컨테이너_포트번호	포트 번호를 지정함
-v 호스트_디스크:컨테이너_디렉터리	볼륨을 마운트함
--net=네트워크_이름	컨테이너를 네트워크에 연결함
-e 환경변수_이름=값	환경변수를 설정함
-d	백그라운드로 실행함
-i	컨테이너와 대화형 모드로 연결함
-t	컨테이너에 터미널 연결함
-help	사용 방법 안내 메시지를 출력함

-p는 --publish, -v는 --volume, -e는 --env, -d는 --detach, -i는 --interactive, -t는 --tty의 생략형이다.

외부와 접속을 위해 통신포트 설정

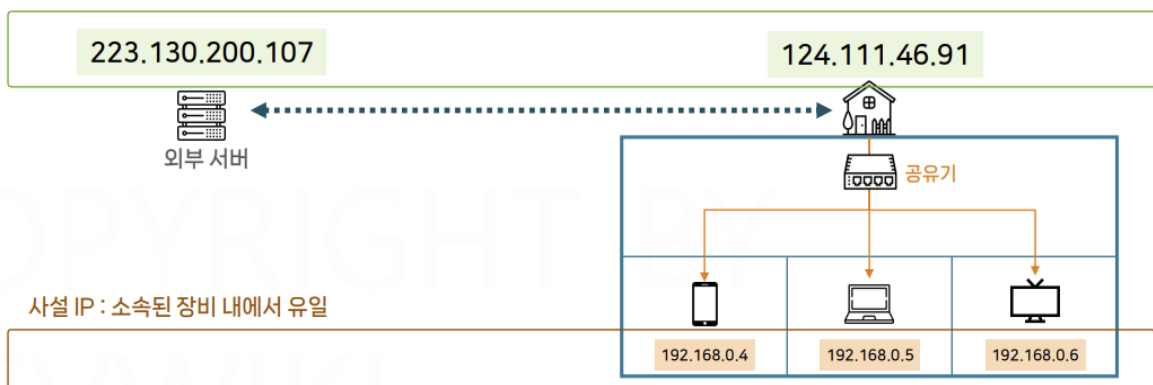


여러 개의 웹 서버 컨테이너 구동 시 호스트_포트번호는 다르게 구성



[공인 IP 와 사설 IP]

공인 IP : 전 세계에서 유일



사설 IP 주소 대역

10.0.0.0 - 10.255.255.255 (Class A)

172.16.0.0 - 172.31.255.255 (Class B)

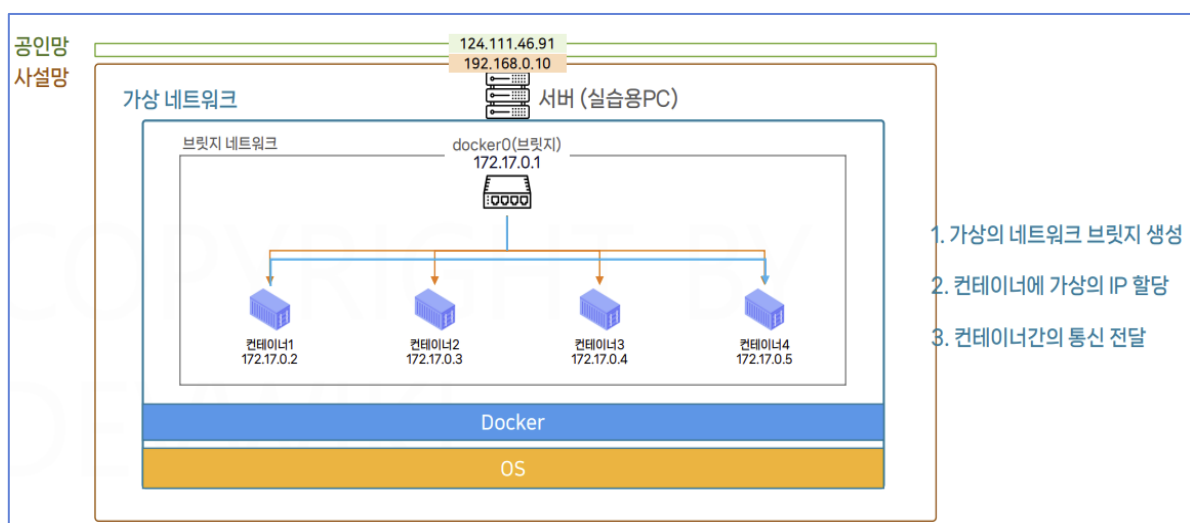
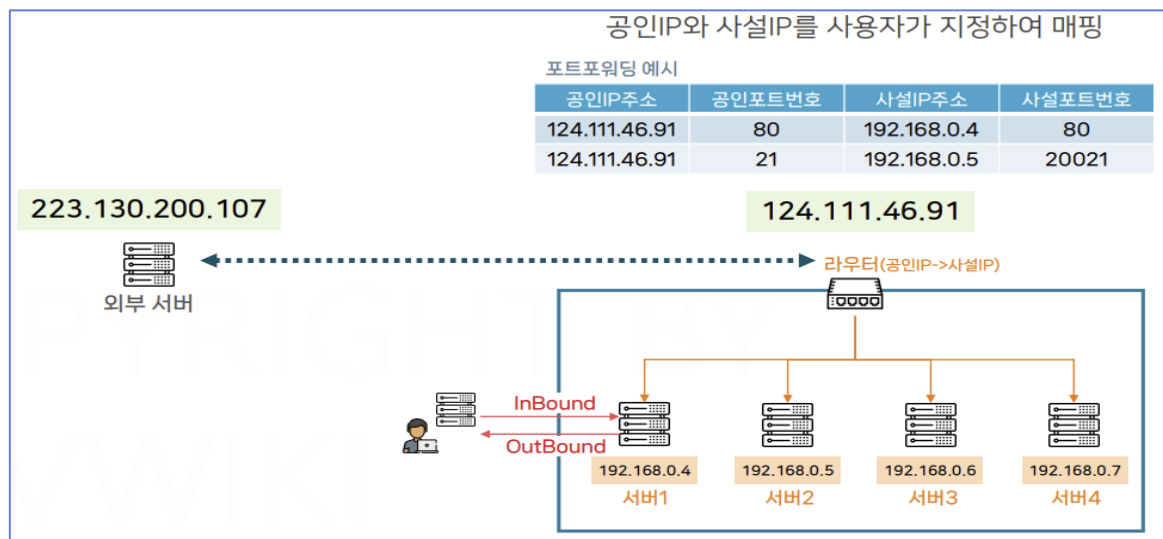
192.168.0.0 - 192.168.255.255 (Class C)

[NAT 와 포트포워딩]

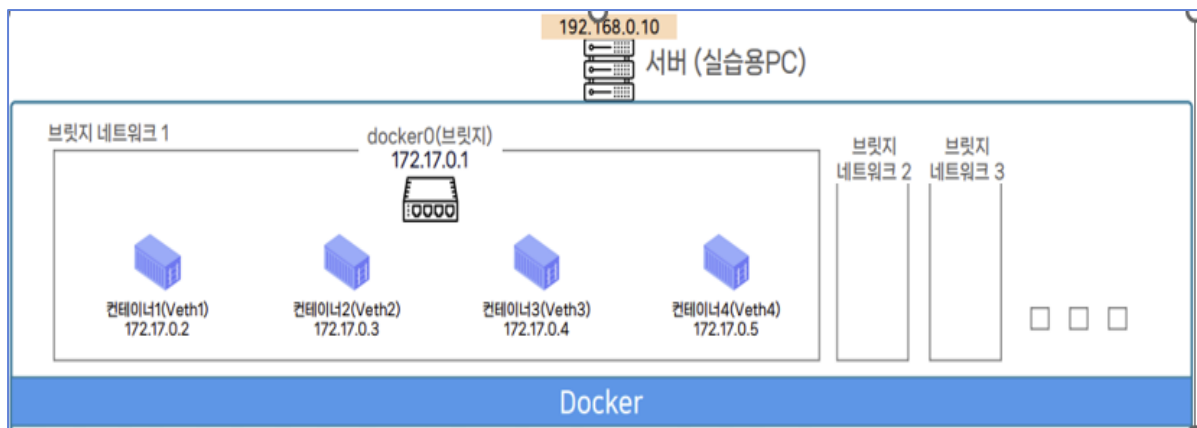
NAT(Network Address Translation)는 Network 계층의 주소(IP)를 변환시켜주는 기술이다.

컨테이너 기반 가상화에서의 포트 포워딩(Port Forwarding)은 호스트와 컨테이너 사이에서 네트워크 트래픽을 전달하는 기술이다. 컨테이너는 가상화된 환경에서 실행되는 독립적인 프로세스로, 컨테이너 내부에서 실행되는 애플리케이션은 호스트와 같은 IP 주소나 포트에 직접적으로 접근할 수 없다. 이를 해결하기 위해 **컨테이너의 포트와 호스트의 포트를 매핑**해주는 것이 포트 포워딩이다.

포트 포워딩은 NAT를 지원하는 장치에서 설정 가능하며 특정 포트를 Open하고 해당 포트로 들어오는 모든 패킷을 내부의 사설 IP와 포트로 전달해주는 기능이다.



브릿지 네트워크(Bridge) : 도커브릿지를 활용해 컨테이너간 통신, NAT 및 포트포워딩 기술을 활용해 외부 통신 지원한다.



docker network ls

네트워크 리스트 조회

docker network inspect 네트워크명

네트워크 상세 정보 조회

docker network create 네트워크명

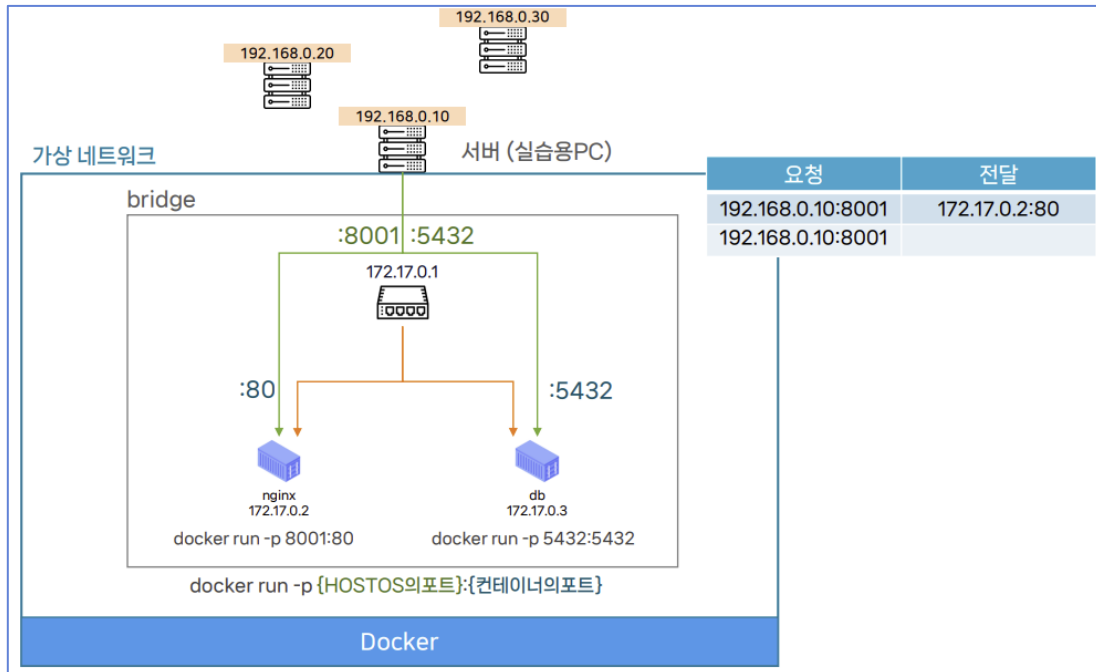
네트워크 생성

docker network rm 네트워크명

네트워크 삭제

docker run -p HostOS의포트:컨테이너의포트

HostOS로의 네트워크 접근을 컨테이너로 포트포워딩



도커 네트워크 생성/삭제

- 도커는 네트워크를 직접 생성하지 않아도 기본 네트워크가 자동으로 세 개 만들어짐
- 네트워크를 명시적으로 지정하지 않고 Docker 컨테이너를 시작하면 기본값인 "bridge" 네트워크로 Docker 컨테이너를 시작 함

```
C:\Windows\System32\cmd
C:\education_deploy>docker network ls
NETWORK ID          NAME       DRIVER  SCOPE
6b0292e00702        bridge    bridge   local
50bb98c7117c        host      host      local
3afbccec352d        none      null      local
C:\education_deploy>
```

브릿지 네트워크(Bridge) : 도커 브릿지를 활용해 컨테이너간 통신, NAT 및 포트포워딩 기술을 활용해 외부 통신 지원
 호스트 네트워크(Host) : 호스트의 네트워크를 공유, 모든 컨테이너는 호스트 머신과 동일한 IP를 사용, 포트 중복 불가능

- 컨테이너에 의해 기동된 일부 서비스를 관리의 용이성을 위해서가 아니라서 기존의 네트워크를 정의하여 사용할 수 있다.

#네트워크 생성 명령

docker network create 네트워크이름

#네트워크 삭제 명령

docker network rm 네트워크이름

- 기타 네트워크 관련 명령

주요 하위 커맨드

커맨드	내용	생략 가능 여부	주요 옵션
connect	네트워크에 컨테이너를 새로이 접속	X	거의 사용하지 않음
disconnect	네트워크에서 컨테이너의 접속을 끊음	X	거의 사용하지 않음
create	네트워크를 생성	X	거의 사용하지 않음
inspect	네트워크의 상세 정보를 확인	X	거의 사용하지 않음
ls	네트워크의 목록을 확인	X	거의 사용하지 않음
prune	현재 아무 컨테이너도 접속하지 않은 네트워크를 모두 삭제	X	거의 사용하지 않음
rm	지정한 네트워크를 삭제	X	거의 사용하지 않음

여러 컨테이너 간의 통신을 위해서는 같은 네트워크에 존재해야 통신 할 수 있다.

예를 들어, tomcat서버 컨테이너 에서 실행 중인 웹 프로젝트가 mysql서버 컨테이너의 데이터베이스를 연동 하려면 컨테이너를 생성 할 때 동일 네트워크로 생성해야 한다.

이 부분의 실습은 데이터저장방식 마운트를 학습 한 후 실습 하도록 하자.

실습하기 02- 포트포워딩을 통해 컨테이너 내부의 웹 페이지 요청

cmd창에서

docker run --name mynginx01 -p 8000:80 -d nginx

```
C:\Users\WKOSTA>docker run --name mynginx01 -p 8000:80 -d nginx
16b5155d49676976d48c22b08079cb0250e90f8bced61256fa59c9585c9dd6c
```

docker images

```
C:\Users\WKOSTA>docker images
REPOSITORY TAG IMAGE ID CREATED SIZE
nginx latest 124b44bfc9cc 7 weeks ago 279MB
```

docker ps -a

```
C:\Users\WKOSTA>docker ps -a
CONTAINER ID   IMAGE     COMMAND                  CREATED    STATUS      PORTS               NAMES
16b5155d4967   nginx     "/docker-entrypoint...." 43 seconds ago Up 42 seconds 0.0.0.0:8000->80/tcp mynginx01
```

브라우저를 열어서 <http://localhost:8000/>

**컨테이너 삭제하기**

```
Microsoft Windows [Version 10.0.19045.5608]
(c) Microsoft Corporation. All rights reserved.

C:\Users\WKOSTA>docker container ps -a
CONTAINER ID   IMAGE     COMMAND                  CREATED    STATUS      PORTS               NAMES
16b5155d4967   nginx     "/docker-entrypoint...." 10 minutes ago Up 10 minutes 0.0.0.0:8000->80/tcp mynginx01

C:\Users\WKOSTA>docker rm 16 → 실행중인 컨테이너 삭제 불가
Error response from daemon: cannot remove container "/mynginx01": container is running: stop the container before removing or force remove

C:\Users\WKOSTA>docker stop 16
16
```

```
C:\Users\WKOSTA>docker ps -a
CONTAINER ID   IMAGE     COMMAND                  CREATED    STATUS      PORTS               NAMES
16b5155d4967   nginx     "/docker-entrypoint...." 12 minutes ago Exited (0) About a minute ago mynginx01

C:\Users\WKOSTA>docker rm 16b
16b

C:\Users\WKOSTA>docker ps -a
CONTAINER ID   IMAGE     COMMAND                  CREATED    STATUS      PORTS               NAMES
```

이미지 삭제

컨테이너를 삭제해도 이미지는 남아있으므로 불필요한 이미지는 즉시 삭제한다.

해당 이미지의 컨테이너가 남아 있으면 이미지 삭제 안된다.

#이미지 삭제 명령어

이미지 삭제 명령어

docker image rm 이미지명 [, 이미지명,...]

docker rmi 이미지명 [, 이미지명,...]

이미지 조회 명령어

docker image ls

실습하기 03(교재 110 ~ 113page)

컨테이너 생성, 실행, 상태 확인 및 종료



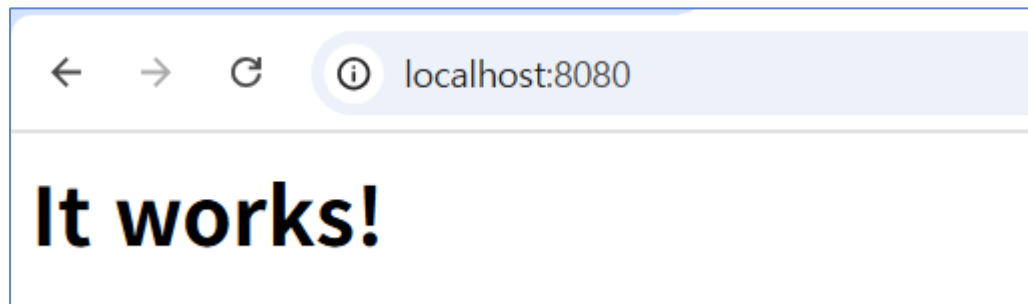
• 컨테이너 정보

옵션 항목

항목	값	항목	내용
컨테이너 이름	apa000ex2	--name apa000ex2	apa000ex2라는 이름으로 컨테이너를 생성
이미지 이름	httpd	-d	백그라운드로 실행
포트 설정	8080:80	-p 8080:80	호스트의 포트 8080을 컨테이너 포트 80으로 포워딩
		httpd	아파치의 이미지 이름. 버전을 지정하지 않았으므로 최신 버전(latest)이 사용된다.

```

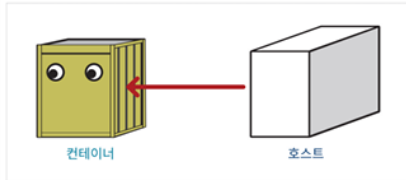
C:\Users\KOSTA>docker run -p 8080:80 --name apa000ex2 -d httpd
Unable to find image 'httpd:latest' locally
latest: Pulling from library/httpd
4ceeea7b3d76: Pull complete
ba78a05e3b3c: Pull complete
0ff470512d2f: Pull complete
4f4fb700ef54: Pull complete
10d01782dc02: Pull complete
Digest: sha256:c11efd67f6308f2c25965e4e9d13ded15e7c45c0367b95f619a16e03c6c1e2b1
Status: Downloaded newer image for httpd:latest
7ac79ee5684515f62b34c3a9d9a00a9207f0d48e192221f92c07c8a91d22ba5b
  
```

브라우저에서 확인하기 - <http://localhost:8080/>

#파일복사 명령어

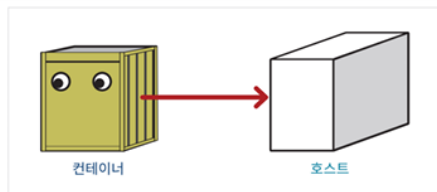
1) Host PC에 있는 파일을 컨테이너안으로 복사하기

```
docker cp 호스트_경로 컨테이너_이름:컨테이너_경로
```



2) 컨테이너안에 있는 파일을 Host PC로 복사하기

```
docker cp 컨테이너_이름:컨테이너_경로 호스트_경로
```



호스트 경로의 예

경로의 예

항목	값
문서 폴더(윈도우)	C:\Users\사용자명\Documents\파일명
문서 폴더(macOS)	/Users/사용자명/Documents/파일명
리눅스의 홈 디렉터리	/home/사용자명/파일명

httpd 서버 기본경로 = /usr/local/apache2/htdocs/

nginx 서버 기본 경로 = /usr/share/nginx/html

tomcat 서버 기본 경로 = /usr/local/tomcat/webapps

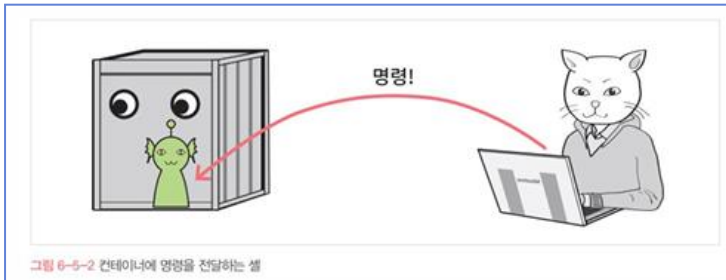
컨테이너 안으로 들어가서 쉘 기반으로 명령어 실행해보기

- docker exec 명령

```
docker exec [옵션] 컨테이너_이름 /bin/bash
```

- docker run 명령

```
docker run [옵션] 이미지_이름 /bin/bash
```

#docker exec 명령

예) **docker exec -it apa000ex23 /bin/bash**

#docker run 명령

예) **docker run --name apa000ex23 -it -p 8089:80 httpd /bin/bash**

-i(--interactive) - 컨테이너의 표준 입력(stdin)을 활성화. (주로 -it 함께 사용)

-t(--tty) - tty(가상 터미널)를 할당.

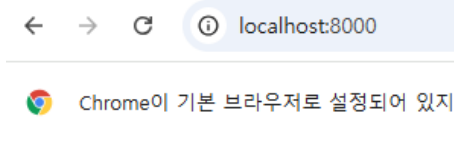
리눅스에 키보드를 통해 표준 입력(stdin)이 전달할 수 있게 한다. (주로 -it 함께 사용)

실습하기 04 - cp 명령어를 이용하여 웹 페이지 변경하기

#httpd 컨테이너를 실행한다.

docker run --name my_httpd -p 8000:80 -d httpd

-브라우저에서 접속하여 확인한다.



It works!

-#httpd 컨테이너 내부로 들어간다.(bin/bash)

docker exec -it my_httpd /bin/bash

```
C:\Users\WKOSTA>docker exec -it my_httpd /bin/bash
root@2f35d2aa128b:/usr/local/apache2#
```

- 현재위치 확인한다.

pwd

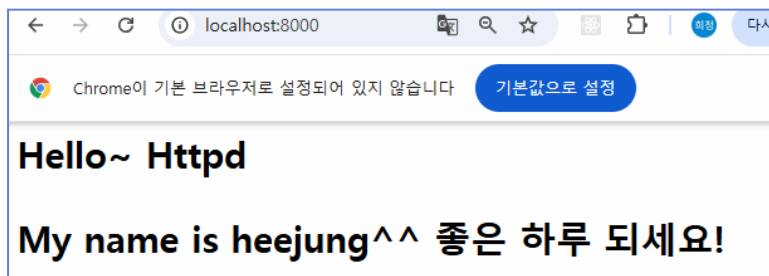
```
root@2f35d2aa128b:/usr/local/apache2# pwd
/usr/local/apache2
```

httpd의 index.html문서를 확인 한후 exit로 컨테이너를 빠져나온다.

```
root@2f35d2aa128b:/usr/local/apache2# ls
bin build cgi-bin conf error htdocs icons include logs modules
root@2f35d2aa128b:/usr/local/apache2# cd htdocs
root@2f35d2aa128b:/usr/local/apache2/htdocs# ls
index.html
root@2f35d2aa128b:/usr/local/apache2/htdocs# cat index.html
<html><body><h1>It works!</h1></body></html>
root@2f35d2aa128b:/usr/local/apache2/htdocs# exit
exit
```

-host pc에서 index.html문서를 작성 한 후 /usr/local/apache2/htdocs/로 복사한다.

```
C:\Users\WKOSTA>docker cp ./index.html my_httpd:/usr/local/apache2/htdocs/
Successfully copied 2.05kB to my_httpd:/usr/local/apache2/htdocs/
```



컨테이너에 있는 파일을 bash실행으로 수정하려면 nano or vi에디터가 필요하다.

bash를 실행하면 기본적으로 에디터가 없기 때문에 설치가 필요하다.

apt-get update

```
root@2f35d2aa128b:/usr/local/apache2/htdocs# apt-get update
Get:1 http://deb.debian.org/debian bookworm InRelease [151 kB]
```

apt-get install nano

```
root@2f35d2aa128b:/usr/local/apache2/htdocs# apt-get install nano
Reading package lists... Done
```

nano index.html

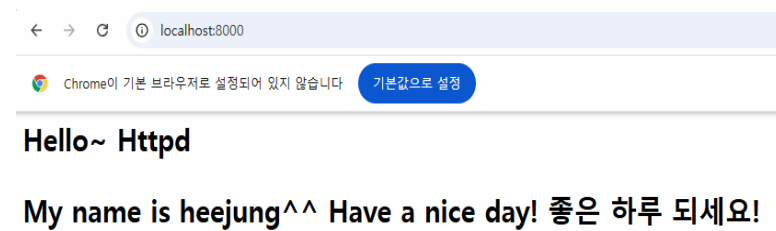
```

GNU nano 7.2 index.html
<html><body><h1>
Hello~ Httpd <p>
My name is heejung^^
Have a nice day!
!
</h1></body></html>
^G Help      ^O Write Out ^W Where Is  ^K Cut       ^T Execute
^X Exit      ^R Read File ^M Replace   ^U Paste     ^J Justify

```

Ctrl +o : 저장

Ctrl +x : 나가기



docker stop 컨테이너이름

docker rm 컨테이너이름

docker rmi 이미지이름

실습하기05 - 하나의 이미지로 여러 개의 서버 실행하기



[요구사항]

nginx 웹서버 container를 3개

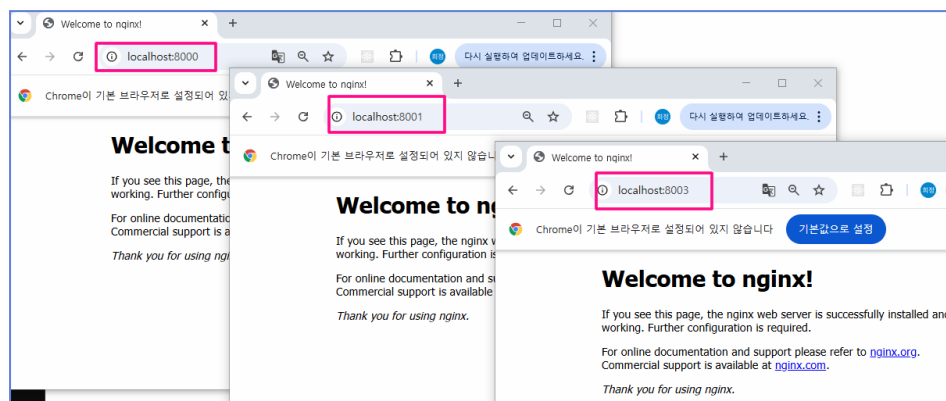
host port를 8000, 8001, 8003 로

name은 nginx01, nginx02, nginx03

백그라운드로 실행 시켜보자.

```
C:\Users\WKOSTA>docker images
REPOSITORY    TAG       IMAGE ID       CREATED        SIZE
nginx         latest    124b44bfc9cc   7 weeks ago    279MB
```

```
C:\Users\WKOSTA>docker container ps -a
CONTAINER ID   IMAGE     COMMAND                  CREATED        STATUS        PORTS                    NAMES
8bd019bdaf69   nginx    "/docker-entrypoint...." 38 seconds ago Up 36 seconds    0.0.0.0:8003->80/tcp    mynginx03
c3fa4ac4d30a   nginx    "/docker-entrypoint...." 46 seconds ago Up 44 seconds    0.0.0.0:8001->80/tcp    mynginx02
9e2de19b4b47   nginx    "/docker-entrypoint...." 2 minutes ago  Up 2 minutes   0.0.0.0:8000->80/tcp    mynginx01
```



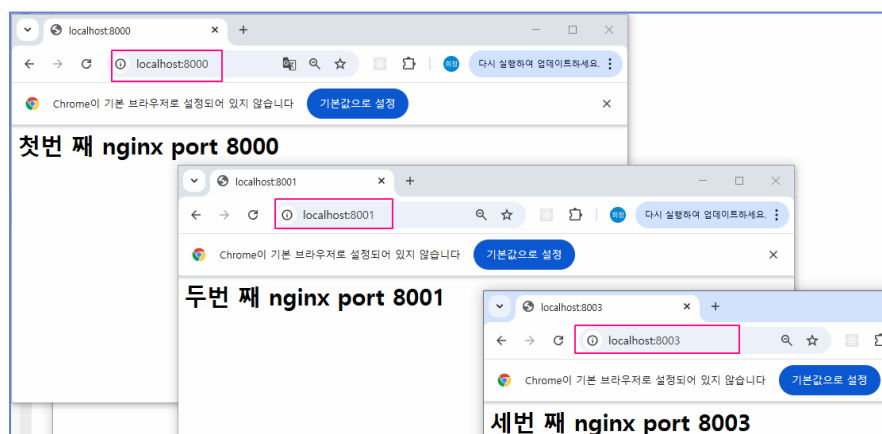
index.html를 생성한 후 각 메시지를 다르게 하여 각 container에 복사한다.

host pc에서 접속했을 때 서로 다른 웹 페이지가 보여지도록 실습한다.

```
C:\Users\WKOSTA>docker cp ./index.html mynginx01:/usr/share/nginx/html/
Successfully copied 2.05kB to mynginx01:/usr/share/nginx/html/

C:\Users\WKOSTA>docker cp ./index.html mynginx02:/usr/share/nginx/html/
Successfully copied 2.05kB to mynginx02:/usr/share/nginx/html/

C:\Users\WKOSTA>docker cp ./index.html mynginx03:/usr/share/nginx/html/
Successfully copied 2.05kB to mynginx03:/usr/share/nginx/html/
```



모든 페이지를 확인 한 후 container를 중지, 삭제를 실행한다.

```
C:\Users\WKOSTA>docker stop 8b
8b

C:\Users\WKOSTA>docker stop c3
c3
```

```
C:\Users\WKOSTA>docker ps -a
CONTAINER ID   IMAGE     COMMAND                  CREATED        STATUS              PORTS          NAMES
8bd019bda6f69  nginx    "/docker-entrypoint..." 8 minutes ago  Exited (0) 34 seconds ago           mynginx03
c3fa4ac4d30a   nginx    "/docker-entrypoint..." 9 minutes ago  Exited (0) 29 seconds ago           mynginx02
9e2de19b4b47   nginx    "/docker-entrypoint..." 10 minutes ago Exited (0) 3 seconds ago           mynginx01

C:\Users\WKOSTA>docker rm 8b
8b

C:\Users\WKOSTA>docker rm c3
c3

C:\Users\WKOSTA>docker rm 9e
9e

C:\Users\WKOSTA>docker ps -a
CONTAINER ID   IMAGE     COMMAND                  CREATED        STATUS        PORTS          NAMES
```

nginx이미지 삭제한다.

```
C:\Users\WKOSTA>docker ps -a
CONTAINER ID   IMAGE     COMMAND                  CREATED        STATUS        PORTS          NAMES

C:\Users\WKOSTA>docker images
REPOSITORY    TAG       IMAGE ID       CREATED        SIZE
nginx         latest   124b44bfc9cc   7 weeks ago   279MB

C:\Users\WKOSTA>docker rmi nginx
Untagged: nginx:latest
Deleted: sha256:124b44bfc9ccd1f3cedf4b592d4d1e8bddb78b51ec2ed5056c52d3692baebc19

C:\Users\WKOSTA>docker images
REPOSITORY    TAG       IMAGE ID       CREATED        SIZE

C:\Users\WKOSTA>
```

다양한 유형의 컨테이너

리눅스 운영체제가 담긴 컨테이너의 종류

이미지 이름	컨테이너의 내용	컨테이너 실행에 주로 사용되는 옵션 및 인자
ubuntu	우분투	-d 없이 -it 옵션만 사용. 인자로는 /bin/bash 등 셸 명령어를 지정한다.
centos	CentOS	-d 없이 -it 옵션만 사용. 인자로는 /bin/bash 등 셸 명령어를 지정한다.
debian	데비안	-d 없이 -it 옵션만 사용. 인자로는 /bin/bash 등 셸 명령어를 지정한다.
fedora	페도라	-d 없이 -it 옵션만 사용. 인자로는 /bin/bash 등 셸 명령어를 지정한다.
busybox	BizyBox	-d 없이 -it 옵션만 사용. 인자로는 /bin/bash 등 셸 명령어를 지정한다.
alpine	알파인 리눅스	-d 없이 -it 옵션만 사용. 인자로는 /bin/bash 등 셸 명령어를 지정한다.

웹 서버 및 데이터베이스 서버용 컨테이너의 종류

이미지 이름	컨테이너의 내용	컨테이너 실행에 주로 사용되는 옵션 및 인자
httpd	Apache	-d로 백그라운드로 실행. -p로 포트 번호 지정
nginx	Nginx	-d로 백그라운드로 실행. -p로 포트 번호 지정
mysql	MySQL	-d를 사용. 실행 시 -e MYSQL_ROOT_PASSWORD와 같이 루트 패스워드를 지정
postgres	PostgreSQL	-d를 사용. 실행 시 -e POSTGRES_ROOT_PASSWORD와 같이 루트 패스워드를 지정
mariadb	MariaDB	-d를 사용. 실행 시 -e MYSQL_ROOT_PASSWORD와 같이 루트 패스워드를 지정

프로그래밍 언어의 런타임 컨테이너의 종류

이미지 이름	컨테이너의 내용	컨테이너 실행에 주로 사용되는 옵션 및 인자
openjdk	자바 런타임	-d를 사용하지 않고, 인자로 java 명령 등을 지정해 도구 형태로 사용한다.
python	파이썬 런타임	-d를 사용하지 않고, 인자로 python 명령 등을 지정해 도구 형태로 사용한다.
php	PHP 런타임	웹 서버가 포함된 것과 실행 명령만 포함된 것으로 나뉘어 제공된다.
ruby	루비 런타임	웹 서버가 포함된 것과 실행 명령만 포함된 것으로 나뉘어 제공된다.
perl	펄 런타임	-d를 사용하지 않고, 인자로 perl 명령 등을 지정해 도구 형태로 사용한다.
gcc	C/C++ 컴파일러	-d를 사용하지 않고, 인자로 gcc 명령 등을 지정해 도구 형태로 사용한다.
node	Node.js	-d를 사용하지 않고, 인자로 app 명령 등을 지정해 도구 형태로 사용한다.
registry	도커 레지스트리	-d 옵션을 사용해 백그라운드로 실행하며, -p 옵션으로 포트 번호를 지정한다.
wordpress	WordPress	-d 옵션을 사용해 백그라운드로 실행하며, -p 옵션으로 포트 번호를 지정한다. MySQL 또는 MariaDB가 필요하다. 접속에 필요한 패스워드는 -e 옵션으로 지정한다.
nextcloud	NextCloud	-d 옵션을 사용해 백그라운드로 실행한다. -p 옵션으로 포트 번호를 지정한다.
redmine	Redmine	-d 옵션을 사용해 백그라운드로 실행하며, -p 옵션으로 포트 번호를 지정한다. PostgreSQL 또는 MySQL이 필요하다.

MySQL 컨테이너 실행 실습하기

항목	값	항목	내용
컨테이너 이름	mysql000ex7	--name mysql000ex7	mysql000ex7이라는 이름으로 컨테이너를 생성
이미지 이름	mysql	-dit	백그라운드에서 실행 및 키보드를 통해 컨테이너 내부의 파일 시스템을 조작
MySQL 루트 패스워드	myrootpass	-e MYSQL_ROOT_PASSWORD=	MySQL의 루트 패스워드를 지정
		mysql	MySQL 이미지 이름. 버전을 지정하지 않았으므로 최신 버전(latest)이 사용된다.

hub.docker.com > Database선택 > mysql검색 > How to use this images

=> https://hub.docker.com/_/mysql

How to use this image

Start a mysql server instance

Starting a MySQL instance is simple:

```
$ docker run --name some-mysql -e MYSQL_ROOT_PASSWORD=my-secret-pw -d mysql:tag
```

... where `some-mysql` is the name you want to assign to your container, `my-secret-pw` is the password to be set for the MySQL root user and `tag` is the tag specifying the MySQL version you want. See the list above for relevant tags.

Connect to MySQL from the MySQL command line client

The following command starts another `mysql` container instance and runs the `mysql` command line client against your original `mysql` container, allowing you to execute SQL statements against your database instance:

```
$ docker run -it --network some-network --rm mysql mysql -hsome-mysql -uexample-user -p
```

... where `some-mysql` is the name of your original `mysql` container (connected to the `some-network` Docker network).

This image can also be used as a client for non-Docker or remote instances:

```
$ docker run -it --rm mysql mysql -hsome.mysql.host -usome-mysql-user -p
```

mysql 컨테이너 실행

docker run --name mysql01 -e MYSQL_ROOT_PASSWORD=1234 -d mysql:8.4

```
C:\Edu\W\docker\data>docker run --name mysql01 -e MYSQL_ROOT_PASSWORD=1234 -d mysql:8.4
Unable to find image 'mysql:8.4' locally
8.4: Pulling from library/mysql
970f697f30e8: Pull complete
f56ec30544fc: Pull complete
8389b884d3d6: Pull complete
35a745903ff9: Pull complete
46d5b31c795a: Pull complete
806f49275cbf: Pull complete
69718387e824: Pull complete
d3dc946e9b73: Pull complete
7a99a8dca35c: Pull complete
a49b4bec6f69: Pull complete
Digest: sha256:5367102acfe9eaa47eb0eb57c8d4f8b96c8c14004859131eac9bbfaa62f81e34
Status: Downloaded newer image for mysql:8.4
e4c5f549142b048a91a1b3fde7ccfd24b039aa81c82b8429621e303d78a32a1f
```

mysql 컨테이너 안으로 들어가기

docker exec -it mysql01 /bin/bash

#mysql 접속하기

mysql -uroot -p1234

```
C:\Users\WKOSTA>docker exec -it mysql01 /bin/bash
bash-5.1# mysql -uroot -p1234
mysql: [Warning] Using a password on the command line interface can be insecure.
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 9
Server version: 8.4.4 MySQL Community Server - GPL

Copyright (c) 2000, 2025, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql>
```

```
Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> create database mytest;
Query OK, 1 row affected (0.03 sec)

mysql> use mytest;
Database changed
mysql> create table emp(empno int primary key, ename varchar(20));
Query OK, 0 rows affected (0.08 sec)

mysql> select * from emp;
Empty set (0.01 sec)

mysql>
```

```
mysql> exit
Bye
bash-5.1# exit;
exit
C:\Users\WKOSTA>
```

위 mysql 실습 후 생성된 container를 중지하고 삭제 한 후 다시 실행 시켜본다.

```
docker run --name mysql01 -d -e MYSQL_ROOT_PASSWORD=1234 mysql:8.4
```

```
docker ps -a
```

```
docker exec -it mysql01 /bin/bash
```

```
mysql -uroot -p1234
```

```
use mytest;
```

```
mysql> use mytest;
ERROR 1049 (42000): Unknown database 'mytest'
```


컨테이너를 삭제 하고 다시 실행했을 때 기존에 생성했던 데이터베이스도 함께 삭제되어 정보가 모두 사라졌다. **때문에 물리적 서버의 디스크를 마운트하여 저장해야 한다.**

(01_Docker의 특징 및 동작원리.pdf 19page참고)

Docker 컨테이너의 저장소

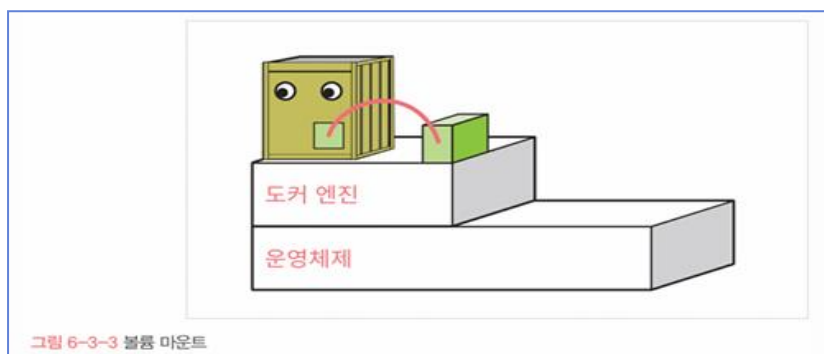
Docker 컨테이너(container)에 쓰여진 데이터는 컨테이너가 삭제될 때 함께 사라지게 된다. Docker에서 돌아가는 많은 애플리케이션이 컨테이너의 생명 주기와 관계없이 데이터를 영속적으로 저장할 해야 하는 경우, 그리고 여러 개의 Docker 컨테이너가 하나의 저장 공간을 공유해서 데이터를 읽거나 써야 하는 경우가 있다. 이렇게 Docker 컨테이너의 생명 주기와 관계없이 데이터를 영속적으로 저장할 수 있도록 Docker는 두가지 옵션을 제공하는데 Docker **볼륨(volume)**과 **바인드 마운트(bind mount)**가 있다.

1)볼륨 마운트

도커 엔진이 관리하는 영역 내에 만들어진 볼륨을 컨테이너에 디스크 형태로 마운트하여 사용하며 직접 조작이 어려우므로 주로 '임시 목적' 또는 '자주 쓰지는 않지만 지우면 안되는 파일을 저장'하는 목적으로 사용한다

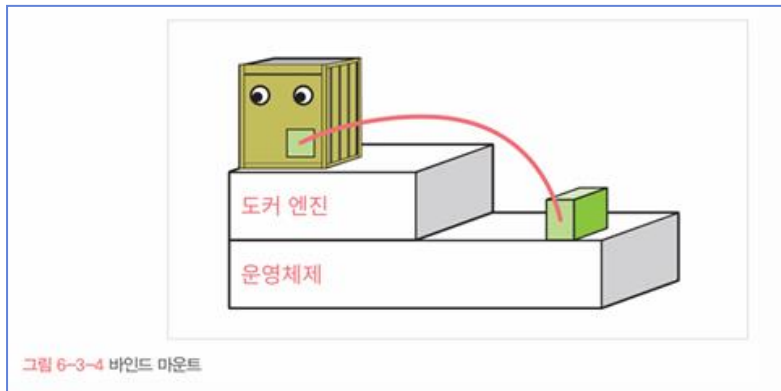
도커 엔진에 의해 관리

- 리눅스 경우 /var/lib/docker/volumns
- 도커 데스크톱의 경우 도커 엔진의 관리영역



2)바인드 마운트

도커 엔진에서 관리하지 않는 영역의 디렉토리를 컨테이너에 마운트하는 방식으로 해당 디렉토리를 직접 조회할 수 있으므로 자주 사용하는 파일을 보관하는 용도로 사용한다.



호스트에서 자주 파일의 내용을 들여다 보고 수정해야 한다면 **바인드마운트**를 사용하고 도커 엔진에 의해 안전하게 관리하고 싶다면 **볼륨마운트**를 사용한다.

볼륨 마운트와 바인드 마운트의 차이

항목	볼륨 마운트	바인드 마운트
스토리지 영역	볼륨	디렉터리 또는 파일
물리적 위치	도커 엔진의 관리 영역	어디든지 가능
마운트 절차	볼륨을 생성한 후 마운트	기존 파일 또는 폴더를 마운트
내용 편집	도커 컨테이너를 통해서	일반적인 파일과 같이
백업	절차가 복잡함	일반적인 파일과 같이

바인드 마운트 명령어(원본파일 또는 디렉토로 먼저생성 필수)

```
docker run ...-v 스토리지실제경로:컨테이너마운트경로
```

볼륨마운트 명령어(볼륨먼저 생성필수)

```
docker volume create 볼륨이름 : 볼륨생성
```

```
docker volume rm 볼륨이름 : 볼륨삭제
```

```
docker run ...-v 볼륨이름:컨테이너마운트경로
```

바인드 마운트 실습하기(교재 186 ~ 189page)



#생성할 컨테이너

항목	값
컨테이너 이름	apa00ex20
이미지 이름	httpd
포트 번호 설정	8090

#마운트할 원본 디렉터리와 마운트 대상

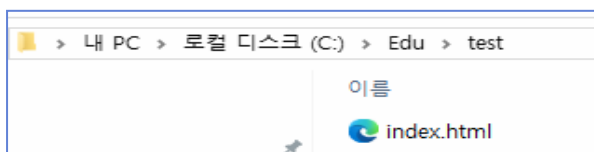
항목	값
컨테이너 마운트 경로 (마운트 대상)	/usr/local/apache2/htdocs
실제 폴더 / 디렉터리 이름 (마운트 원본)	apa_folder
실제 마운트 원본 경로(윈도우)	C:\Users\사용자명\Documents\apa_folder
실제 마운트 원본 경로(macOS)	/Users/사용자명/Documents/apa_folder
실제 마운트 원본 경로(리눅스)	/home/사용자명/apa_folder

자신의 운영체제에 맞는 명령어를 선택

- C:\WEDU\에 test폴더를 생성한다.

> 내 PC > 로컬 디스크 (C:) > Edu > test

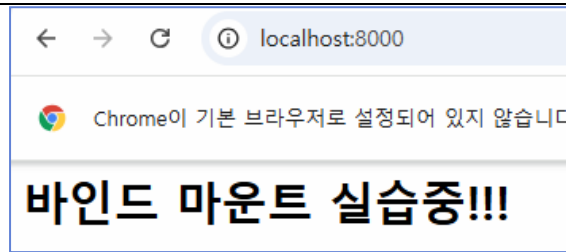
- C:\WEDU\test 폴더에 index.html 만든다.



- cmd창을 연다

docker run --name apa00ex20 -p 8000:80 -d -v C:\WEDU\test:/usr/local/apache2/htdocs httpd

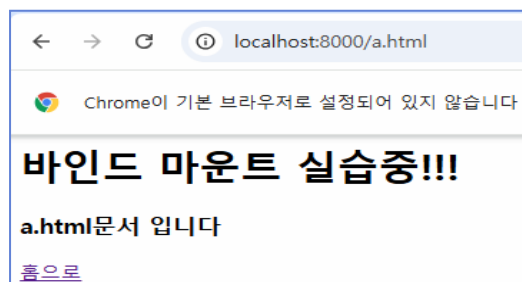
- 브라우저에서 <http://localhost:8000/> 실행해본다.



바인드마운트로 연결되어 있어 직접 파일을 조작 할 수 있다.

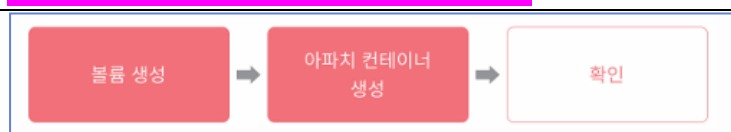
C:\WEduWtestWa.html 작성

```
<html> <body> <h1>
<h1>바인드 마운트 실습중!!!</h1>
<h3>a.html문서 입니다</h3>
<a href="/index.html"> 홈으로 </a>
</h1> </body> </html>
```



실습 완료 후 컨테이너 중지하고 삭제한다.

볼륨마운트 실습하기(교재 190~192페이지)



생성할 컨테이너 정보

항목	값
컨테이너 이름	apa000ex21
이미지 이름	httpd
볼륨 이름	apa000vol1(마지막은 소문자 L 뒤에 숫자 1이다)
포트 번호 설정	8091

마운트 할 볼륨 대상**마운트 원본과 마운트 대상 정보**

항목	값
컨테이너 마운트 경로	/usr/local/apache2/htdocs
마운트 원본 볼륨	apa000vol1

- 볼륨 생성**docker volume create apa000vol1****- 볼륨 확인****docker volume ls**

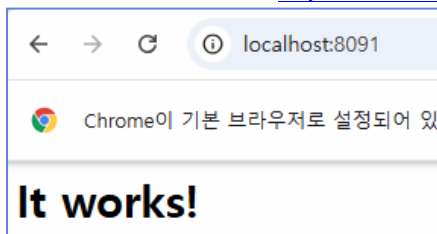
```
C:\Users\KOSTA>docker volume ls
DRIVER      VOLUME NAME
local       3a7a2602aaa0c763f7a87d9b98510a0a7a428570178ba735c687c80938d81e2e
local       apa000vol1
local       e2258e56b90ee32e13c37aec3990b2339b8578387708113684e859b768f37ca6
```

- 볼륨의 상세정보 출력**docker volume inspect apa000vol1**

```
C:\Users\KOSTA>docker volume inspect apa000vol1
[
  {
    "CreatedAt": "2025-03-30T07:43:46Z",
    "Driver": "local",
    "Labels": null,
    "Mountpoint": "/var/lib/docker/volumes/apa000vol1/_data",
    "Name": "apa000vol1",
    "Options": null,
    "Scope": "local"
  }
]
```

- 볼륨 연결하면서 컨테이너 생성하기**docker run -d --name apa000ex21 -p 8091:80 -v apa000vol1:/usr/local/apache2/htdocs httpd**

```
C:\Users\KOSTA>docker run -d --name apa000ex21 -p 8091:80 -d -v apa000vol1:/usr/local/apache2/htdocs httpd
8ed8397ce3a173db4cfb6d692b4e4e6d1c5d7eb167580a85ccbb2cb287fa1509
```

-브라우저로 실행 해본다 -<http://localhost:8091>**- 컨테이너 내부로 들어가기**

```
docker exec -it apa000ex21 /bin/bash
```

- httpd 홈으로 이동하기

```
cd /usr/local/apache2/htdocs/
```

- nano 설치하기

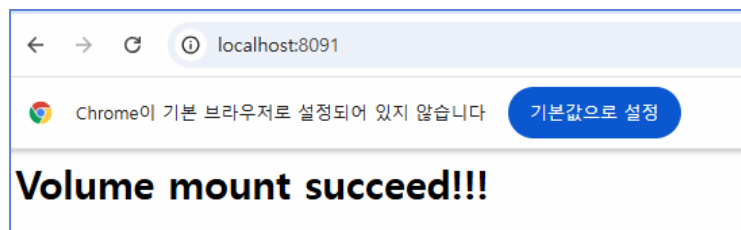
```
apt-get update
```

```
apt-get install nano
```

- nano로 index.html 수정하기

```
nano index.html
```

- 브라우저 실행하기



- exit 로 컨테이너로 빠져 나온 후 컨테이너 중지 후 삭제한다.

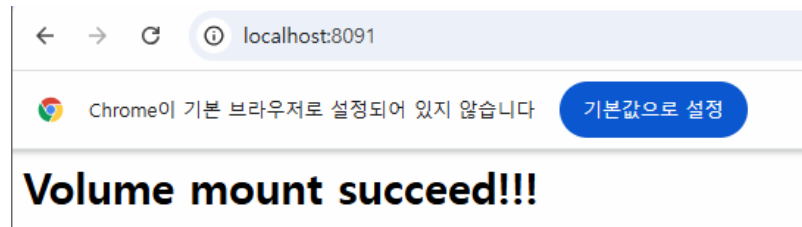
```
C:\Users\KOSTA>docker ps -a
CONTAINER ID   IMAGE     COMMAND                  CREATED        STATUS        PORTS                    NAMES
8ed8397ce3a1   httpd     "httpd-foreground"      9 minutes ago Up 9 minutes   0.0.0.0:8091->80/tcp     apa000ex21

C:\Users\KOSTA>docker stop apa000ex21
apa000ex21

C:\Users\KOSTA>docker rm apa000ex21
apa000ex21

C:\Users\KOSTA>docker ps -a
CONTAINER ID   IMAGE     COMMAND                  CREATED        STATUS        PORTS                    NAMES
C:\Users\KOSTA>
```

- 다시 볼륨을 연결해서 컨테이너를 생성 한 후 브라우저를 요청해보자.



컨테이너를 삭제하고 다시 생성 했어도 index.html 문서의 정보가 저장되어 있다!

실습 완료 후

컨테이너 중지 > 삭제 > 볼륨 삭제 > 이미지 삭제까지 완료한다.

볼륨마운트 Mysql실습하기

앞전의 mysql실습에서 생성한 데이터베이스와 테이블이 컨테이너를 삭제 한 후 다시 실행 했을 때 모든 정보가 사라졌다. 볼륨마운트를 연결하여 컨테이너 삭제 후 다시 실행해도 데이터베이스와 테이블의 정보가 유지 하도록 한다.

- 1)볼륨을 생성한다.
- 2)mysql 컨테이너로 실행한다.(mysql 컨테이너 연결 볼륨디렉토리는 **/var/lib/mysql**)
- 3)mysql 컨테이너 안으로 들어간다.
- 4)mysql에 접속한다.
- 5)데이터베이스 생성한다.
- 6)데이터베이스 선택한다.
- 7)테이블을 생성한다.
- 8) 레코드를 추가한다.
- 9)생성된 정보를 확인하고 컨테이너를 빠져나 온다.
- 10) 컨테이너를 중지 시키고 컨테이너를 삭제한다.
- 11) 컨테이너를 다시 생성 한다.
- 12) 컨테이너 내부로 접속하여 앞에 생성한 DB와 테이블이 있는지 확인한다.
- 13) 모든 테스트가 완료 된 후 컨테이너 중지 > 삭제>볼륨삭제 > 이미지 삭제 한다.

```

C:\Users\KOSTA>docker volume create myvol
myvol

C:\Users\KOSTA>docker volume ls
DRIVER      VOLUME NAME
local       3a7a2602aaa0c763f7a87d9b98510a0a7a428570178ba735c687c80938d81e2e
local       e2258e56b90ee32e13c37aec3990b2339b8578387708113684e859b768f37ca6
local       myvol

C:\Users\KOSTA>docker volume inspect myvol
[
  {
    "CreatedAt": "2025-03-30T08:33:51Z",
    "Driver": "local",
    "Labels": null,
    "Mountpoint": "/var/lib/docker/volumes/myvol/_data",
    "Name": "myvol",
    "Options": null,
    "Scope": "local"
  }
]

```

```

C:\Users\KOSTA>docker run -d --name mysql01 -v myvol:/var/lib/mysql -e MYSQL_ROOT_PASSWORD=1234 mysql:8.4
Unable to find image 'mysql:8.4' locally
8.4: Pulling from library/mysql
7ec3ee0f0f08: Download complete
7d08e04e916f: Download complete
779c43f76385: Download complete
804bb8ae89de: Download complete
3d64c2aa182d: Download complete
29a2795e8756: Download complete
12f538368250: Download complete
a78747d9f656: Download complete
c324f526407a: Download complete
2dd85f24b56a: Download complete
Digest: sha256:0f775a92980b41c87c58f934a204de80431dd4d854057160ec1cb936663eabe9
Status: Downloaded newer image for mysql:8.4
ad2cff4e7f013c5e255b681cffe4cbdf7b8843f7400b3e6d031a041dc44aa813

C:\Users\KOSTA>docker ps -a
CONTAINER ID   IMAGE      COMMAND                  CREATED        STATUS        PORTS                               NAMES
ad2cff4e7f01   mysql:8.4   "docker-entrypoint.s..." 24 seconds ago Up 13 seconds 3306/tcp, 33060/tcp            mysql01

C:\Users\KOSTA>docker exec -it mysql01 /bin/bash
bash-5.1# mysql -uroot -p1234
mysql: [Warning] Using a password on the command line interface can be insecure.
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 8
Server version: 8.4.4 MySQL Community Server - GPL

Copyright (c) 2000, 2025, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> create database mydb;
Query OK, 1 row affected (0.02 sec)

mysql> use mydb;
Database changed
mysql> create table member(id varchar(20) primary key, age int );
Query OK, 0 rows affected (0.08 sec)

```



```
mysql> insert into member values('jang', 20);
Query OK, 1 row affected (0.02 sec)

mysql> select * from member;
+-----+-----+
| id    | age  |
+-----+-----+
| jang  | 20   |
+-----+-----+
1 row in set (0.00 sec)

mysql> exit;
Bye
bash-5.1# exit
exit
```

```
C:\Users\KOSTA>docker ps -a
CONTAINER ID   IMAGE     COMMAND                  CREATED        STATUS        PORTS                    NAMES
ad2c1f4e7f01   mysql:8.4 "docker-entrypoint.s..." 7 minutes ago  Up 7 minutes  3306/tcp, 33060/tcp      mysql01

C:\Users\KOSTA>docker stop mysql01
mysql01

C:\Users\KOSTA>docker rm mysql01
mysql01

C:\Users\KOSTA>docker ps -a
CONTAINER ID   IMAGE     COMMAND                  CREATED        STATUS        PORTS                    NAMES

C:\Users\KOSTA>docker volume ls
DRIVER         VOLUME NAME
local          3a7a2602aaa0c763f7a87d9b98510a0a7a428570178ba735c687c80938d81e2e
local          e2258e56b90ee32e13c37aec3990b2339b8578387708113684e859b768f37ca6
local          mysql_vol
local          myvol

C:\Users\KOSTA>docker run --name mysql02 -v myvol:/var/lib/mysql -d -e MYSQL_ROOT_PASSWORD=1234 mysql:8.4
6178baedd9684067d5cd4f098caa2fb8e3747db92243884995383e4544ce44be

C:\Users\KOSTA>docker exec -it 617 /bin/bash
bash-5.1# mysql -uroot -p1234
mysql: [Warning] Using a password on the command line interface can be insecure.
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 8
Server version: 8.4.4 MySQL Community Server - GPL

Copyright (c) 2000, 2025, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> use mydb;
Reading table information for completion of table and column names
You can turn off this feature to get a quicker startup with -A

Database changed
mysql> select * from member;
+-----+-----+
| id    | age  |
+-----+-----+
| jang  | 20   |
+-----+-----+
```

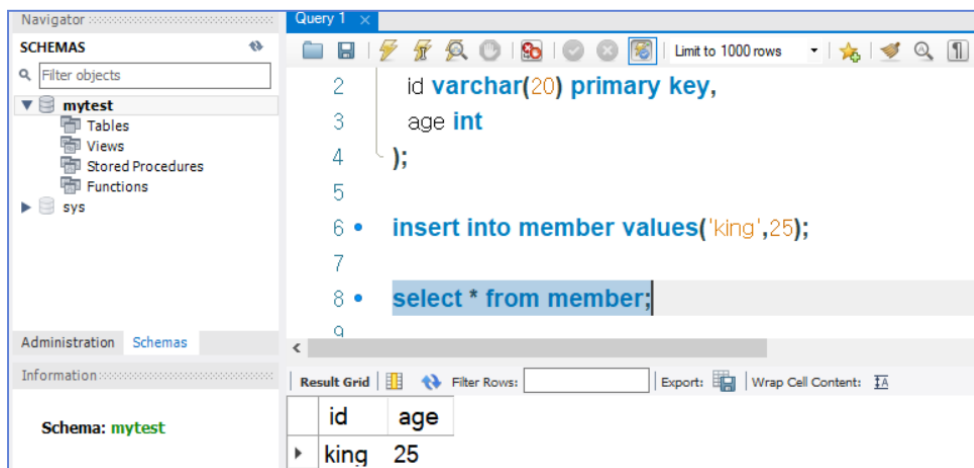
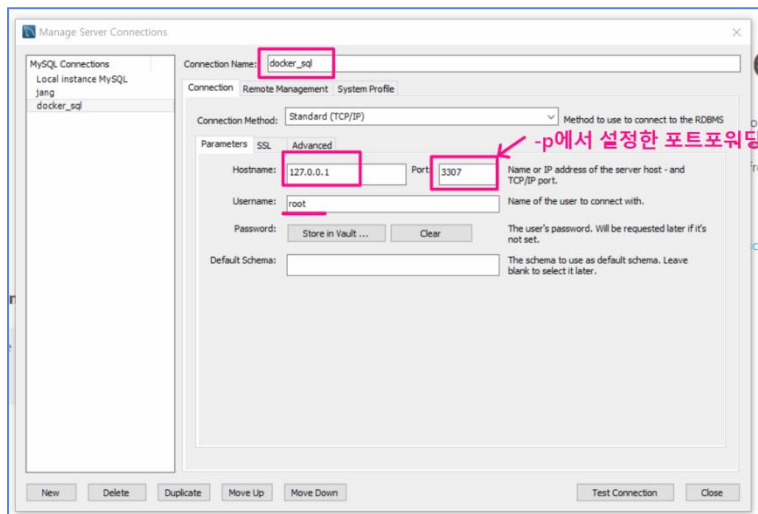
Docker 컨테이너로 실행한 mysql워크벤치로 접속하기

#볼륨생성

docker volume create myvol

#컨테이너 생성(-v, -p, -e 옵션)

```
docker run --name mysql01 -e MYSQL_ROOT_PASSWORD=1234 -e MYSQL_DATABASE=mytest -d -p 3307:3306 -v myvol:/var/lib/mysql mysql:8.4
```



cmd창에서

```

C:\Users\KOSTA>docker exec -it mysql01 /bin/bash
bash-5.1# mysql -uroot -p1234
mysql: [Warning] Using a password on the command line interface can be insecure.
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 10
Server version: 8.4.4 MySQL Community Server - GPL

Copyright (c) 2000, 2025, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> use mytest;
Reading table information for completion of table and column names
You can turn off this feature with -A

Database changed
mysql> select * from member;
+-----+-----+
| id | age |
+-----+-----+
| king | 25 |
+-----+-----+
1 row in set (0.00 sec)

mysql>

```

실습하기(tomcat 10 + mysql 연동) - ex01_myPC_docker_web폴더참고

#네트워크이름 : web_net

03_Docker 명령어 사용하기.pdf 10page 참고

여러 컨테이너 간의 통신을 위해서는 같은 네트워크에 존재해야 통신 할 수 있다.

#My-sql

volume이름 : mysql_vol

mysql 마운트 경로 : /var/lib/mysql

이미지이름 : mysql:8.4

컨테이너이름 : mysql_01

-e 옵션 : MYSQL_ROOT_PASSWORD=1234

MYSQL_DATABASE=todo_db

-p옵션 : 3307:3306

#tomcat

volume이름 : tomcat_vol

tomcat 마운트경로 /usr/local/tomcat/webapps/

이미지이름 : tomcat:10

컨테이너이름 : tomcat_01

-p 옵션 : 80:8080

tomcat서버에 배포 할 웹프로젝트 war파일 준비

Docker 컨테이너 Tomcat에서는 context path가 webapps 아래의 폴더 이름으로 변경된다.
만약 contextpath를 "/"로 지정하고 싶다면 webapps/ROOT 폴더를 만들고 war를 webapps/ROOT폴더에 넣는다.

준비된 war파일을 cp명령어로 /usr/local/tomcat/webapps/ROOT 에 넣는다.

스크립트

컨테이너간의 통신을 위한 네트워크 생성

```
docker network create web_net
```

mysql의 볼륨마운트 생성

```
docker volume create mysql_vol
```

tomcat의 볼륨마운트 생성

```
docker volume create mytomcat_vol
```

#tomcat 서버 기본 경로 = /usr/local/tomcat/webapps

#host pc의 mysql 서비스를 중지시킨다.

mysql pull + create + start

#-e TZ=Asia/Seoul: MySQL 서버의 시간대를 한국 시간, -v /etc/localtime:/etc/localtime:ro: 호스트의 시간 정보를 컨테이너와 공유하여 시스템 시간이 일치하도록 한다.

```
docker run --name my_sql -e MYSQL_ROOT_PASSWORD=1234 -e MYSQL_DATABASE=web_basic -e TZ=Asia/Seoul -v /etc/localtime:/etc/localtime:ro -d -p 3306:3306 -v mysql_vol:/var/lib/mysql --net web_net mysql:8.4
```

https://hub.docker.com/_/mysql 참고

Environment Variables

When you start the `mysql` image, you can adjust the configuration of the MySQL instance by passing one or more environment variables on the `docker run` command line. Do note that none of the variables below will have any effect if you start the container with a data directory that already contains a database: any pre-existing database will always be left untouched on container startup.

See also <https://dev.mysql.com/doc/refman/5.7/en/environment-variables.html> for documentation of environment variables which MySQL itself respects (especially variables like `MYSQL_HOST`, which is known to cause issues when used with this image).

MYSQL_ROOT_PASSWORD

This variable is mandatory and specifies the password that will be set for the MySQL `root` superuser account. In the above example, it was set to `my-secret-pw`.

MYSQL_DATABASE

This variable is optional and allows you to specify the name of a database to be created on image startup. If a user/password was supplied (see below) then that user will be granted superuser access (corresponding to `GRANT ALL`) to this database.

MYSQL_USER , MYSQL_PASSWORD

tomcat pull + create + start

```
docker run --name my_tomcat -p 80:8080 -d -v mytomcat_vol:/usr/local/tomcat/webapps --net web_net tomcat:10
```

프로젝트 ROOT.war 준비하기 - context.xml 설정변경(url, password확인)

/*

```
<Resource name="jdbc/mysql"
  auth="Container"
  type="javax.sql.DataSource"
  driverClassName="com.mysql.cj.jdbc.Driver"
  url="jdbc:mysql://my_sql:3306/web_basic"
  username="root"
  password="admin"
  maxTotal="20"
  maxIdle="10"
  maxWaitMillis="-1"/>
```

```
<Resource name="jdbc/mysql"
  auth="Container"
  type="javax.sql.DataSource"
  driverClassName="com.mysql.cj.jdbc.Driver"
  url="jdbc:mysql://my_sql:3306/web_basic"
  username="root"
  password="1234"
  maxTotal="20"
  maxIdle="10"
  maxWaitMillis="-1"/>
```

*/

tomcat 기본폴더에 ROOT.war 복사

```
docker cp ./ROOT.war my_tomcat:/usr/local/tomcat/webapps
```

mysql bash셸 접속

```
docker exec -it my_sql /bin/bash
```

mysql로그인

```
mysql -uroot -p1234
```

데이터베이스선택

```
use web_basic
```

테이블생성 + 레코드 추가

```
create table todo_list(  
    id int primary key auto_increment,  
    done char(1) default 0,  
    content varchar(100),  
    reg_date datetime default now()  
);  
  
insert into todo_list(content) value('Studying');  
insert into todo_list(content) value('meeting friends');  
insert into todo_list(content) value('Practicing dockers');  
  
select id,done,content, reg_date from todo_list;  
  
commit;  
  
#bash(셸) 빠져나오기  
exit  
exit
```

tomcat bash셸 접속

```
docker exec -it my_tomcat /bin/bash
```

tomcat log정보 확인하기

```
docker logs -f my_tomcat
```

(-f 옵션은 follow의 약자로, 로그를 실시간으로 계속 이어서 보는 옵션)

mysql log정보확인하기

docker logs my_sql

mysql log 새로운 마지막 정보만 확인하기

docker logs --tail 0 -f mysql

#브라우저에서 실행

http://localhost/

브라우저에 실행된 todoList CRUD 해본다.

mysql , tomcat container 모두 stop -> rm 한다.

mysql , tomcat container 를 다시 실행시켜본다.

삭제 후 다시 실행했을 때 정보가 그대로 유지되고 있음을 확인한다.